

Axel Estrada

5/1/26

CMPU-395: Machine Learning

Professor Stapleton

Final Project Rough Draft

Introduction and Motivation

Audio classification is a huge field within machine learning. Applications range from reading sheet music to generating digital audio effects to classifying instruments by various factors. In the guitar world, there are endless combinations of effects a person can use to produce a desired sound and feel. Many amplifier and effects models have been developed through both analog and digital means, and much of the machine-learning work on guitars has focused on how to digitally recreate analog amp and effect models. While these models have gotten so good that they're nearly indistinguishable from their physical counterparts (Bognár 2), not as much work has been done on audio classification of guitar effects.

Classification differs from effects recreation not only in its goal, but in its complexity. The major difference is that classification requires an extensive amount of data on various combinations of both individual amplifiers, effects, and tones on the guitar, as well as in combination with both other instruments and guitars being played at the same time. Research on this topic has been conducted before, both in the context of individual guitar recordings and mixed audio recordings involving other instruments (Hinrichs et al., 2022), though it's quite scarce. Previous research has also been done to create a machine-learning model that can classify electric guitar recordings from different manufacturers and models (de Castro Rabelo Profeta and Schuller 478), with varying degrees of success.

This project is an exploratory analysis for creating and training a machine learning model that can identify electric guitar pedal effects from audio recordings. The guiding research questions for this project were as follows:

1. Can we develop a model that accurately predicts if an electric guitar audio recording is using overdrive, distortion, or fuzz audio effects?

2. What Machine Learning models work best for guitar audio classification?

The initial expectations for this project were that Convolutional Neural Networks would serve as a great starting point for answering these questions. Various papers focused on answering similar research questions utilized CNNs trained with labeled mel-spectrogram representations of audio recordings. Previous research by Hinrichs et al. managed to create a model that achieved up to 97.4% accuracy (Hinrichs et al. 1) for classifying guitar effects while other instruments were playing.

Data Description

The GUITAR-FX-DIST (GFD) and IDMT-SMT-Guitar (ISG) Datasets were the primary datasets used for this project. ISG is the largest publicly available data set of electric guitar and bass recordings. The dataset consists of 55044 WAV files (44.1 kHz, 16bit, mono), with 20592 monophonic guitar notes and 13860 polyphonic guitar sounds. Monophonic audio samples are those that only contain one discrete note at a time. Polyphonic audio samples contain multiple notes at the same time.

ISG guitar recordings are organized into four subsets. The first subset contains different playing techniques, including plucking styles and expression styles. The second subset consists of 400 monophonic and polyphonic note events, each played with two different guitars, with no expression styles. The third subset is five short monophonic and polyphonic guitar recordings. The last subset is 64 short musical pieces grouped by genre.

The GFD dataset is an extension of the ISG dataset created by Marco Comunità. It used the same recordings as ISG but processed with overdrive, distortion, and fuzz audio effects. A total of 14 different pedals were used, and each recording was produced through a MATLAB script he wrote. The dataset is organized into four sets: monophonic discrete, monophonic continuous, polyphonic discrete, and polyphonic continuous. Discrete and continuous describe how the audio samples change over time (i.e., in steps or smooth curves). Each set contained folders with the audio files, organized by the pedal that was used to generate them. Within each folder was a .csv of the parameters used to generate the effects for that sample. The dataset

contains roughly 550k samples (~305 hours) and also 550k mel-specs, and utilizes the same audio format as the ISG dataset. The mel-specs were organized the same as the audio recordings.

Mel-spectrograms are a useful visualization tool for seeing what audio sounds like. It is a time-frequency representation of an audio signal in which the frequency axis is scaled according to how humans generally perceive sound (Roberts, 2020). Humans have an easier time distinguishing between lower frequencies than they do higher frequencies. As such, mel-specs compresses higher frequencies while emphasizing lower frequencies.

Most of my exploration was done with the GFD dataset, as it provided both the audio and mel-specs recordings necessary for training the models, simplifying a large part of the early analysis. The original audio and spectrogram files were not modified throughout the research, so as to ensure training and evaluation consistency. The dataset was remapped using the following pedal-to-effects mapping:

Table 1: Effects-Pedal Mapping

Effect	Pedals
OverDrive	808, TS9, BD2, OD1, SD1, MGS
Distortion	DS1, RAT, DPL, MT2
Fuzz	FFC, BMF, RBM, VTB

All mel-spec folders were reorganized into HDF5 files so that data loading and processing could operate as efficiently as possible. HDF5 stands for ‘The Hierarchical Data Format version 5’. It is an open-source compressed file format meant for large, complex heterogeneous data, which uses a ‘file-directory’ like structure (Wasser, 2026). Within this project, it allowed me to generate ‘shard’ files with long lists of mel-specs and their corresponding effects labelings, which could then be uploaded to Google Colab. Instead of needing to upload thousands of files and then access each one individually, we could upload a few organized files and access each spectrogram/label via list indexing. Reasons why this was necessary are explained in the methods section.

Methods

The code for this project was written in Python as a notebook meant for Google Colab and Jupyter Notebook. Colab was used for three main reasons. The first was because the project required it. The second reason was that the pro version offers access to a variety of different GPU models designed for machine learning training, which would make our training far faster than if we used just a CPU. The third reason was that I could remotely train models without affecting my computer's performance for other tasks.

A majority of this project was spent figuring out how to systematically and efficiently store, organize, and access all of the mel-spec files to then train a model on Colab. The initial idea was to take all of the GFD mel-specs, upload them to Colab, and then perform analysis from there. Unfortunately, this proved to be a far more difficult task than initially thought. The first major issue to arise was that Colab doesn't allow folders to be uploaded directly, only files. This was a problem 1) because of the sheer number of files we had (~ 516,000 samples), and 2) because if a session is left idle for too long, Colab erases all uploaded files, and the process needs to be done again.

The first solution I came up with was to upload all the files to GitHub and then download them remotely. This was immediately shut down after finding out that GitHub only allows a maximum storage of ~2 GB, and GitHub's large file storage (LFS) didn't extend that by much more. The next solution from there was to download the Google Drive Desktop app, and then synchronize the file folders from my local computer to my Google Drive. This process took forever, depending on my available internet connection, but eventually I was able to upload all of the original mel-specs.

Unfortunately, this then led to the next major issue. Google Drive I/O reads and writes are really, really slow. As such, parsing through thousands of files to train one model for even 10 epochs proved difficult, and given that we needed a stable connection the entire time (otherwise we'd lose the model), this proved highly impractical. I thought to copy the files from the synchronized folder to the '/contents' folder of the Colab notebook so that the file access

speeds could be improved, but it also took a long time to perform, and ran into the previous issue of needing to do this every time the session is lost.

This is where the HDF5 files mentioned earlier came in. This file format allowed me to shrink the number of files needed to upload to Colab from ~516,000 down to ~60, depending on how many mel-specs are stored in each file, and also slightly decreased the dataset size (~25 GB down to ~19 GB). This significantly improved the processing speed of the two previous solutions and finally ensured a consistent way to upload our mel-spec dataset to Colab.

Even with this new process, however, training still takes quite a while, and the risk of losing our progress due to network issues was ever high due to my needing to move around a lot across campus. At this time, I also lost access to most of the high-end GPUs available for training (ran out of Colab training tokens), and the available GPUs were either on par with or slower than my laptop GPU. As such, I decided the best method for training a model was overnight, when it could use all of my system's resources.

Due to this series of unfortunate events, not much time could be spent training and testing different model architectures for the task. Two models were trained and evaluated. The first was a simple VGG neural network with 4 convolutional blocks, including a ReLU activation function and a Maxpooling layer at each block. At the end, an adaptive average pooling layer was applied in the off-chance that one of the mel-specs was badly formatted, followed by a flattening layer to map onto our categories.

The other model was a pretrained model called ResNet18 available through the torchvision library, which utilized the ResNet architecture. This architecture features 18 residual blocks, each consisting of two convolutional layers, a batch normalization function, and a ReLU activation function. The key feature of this architecture is the skip connections at each block, which bypass one or more layers (GeeksforGeeks, 2025). The model was modified to work with our dataset and categories, so the first convolutional layer was changed from a 3-channel input to a single channel, and the final layer mapped onto our labels.

Each model utilized Adam as its optimizer and Cross-Entropy-Loss for its loss function. Training and testing were done with a batch size of 128 samples and a learning rate of 0.001.

The test splits used were 80/20 or 90/10, with samples being randomly taken across each of the groupings for an even training distribution.

Results

From the training and testing I was able to run, I achieved the following results:

Table 2: Training results

Model Architecture	Total Epochs	Test/Split	Total Training Time (min.)	Total Model Params.	Overall Accuracy
Simple VGG Model 1 (round 1)	10	80/20	71	625,294	94.91%
Simple VGG Model 1 (round 2)	20	80/20	133	625,294	94.96%
Simple VGG Model 1 (round 3)	30	80/20	213	625,294	94.94%
Simple VGG Model 2	30	80/20	183	625,294	92.34%
ResNet18 Model 1 (Pretrained; round 1)	10	90/10	74	11,171,779	94.66%
ResNet18 Model 1 (Pretrained; round 2)	20	90/10	148	11,171,779	99.3%

Simple VGG Model 1 was trained over 3 rounds, with 10 epochs in each round, and achieved an overall accuracy of 94.94%. The training/test split was not changed across each round. This likely resulted in an overfitting of the data, given the general lack of increase in accuracy across each round. Simple VGG Model 2 was trained in only one round, utilizing 30 epochs, and achieved an overall accuracy of 92.34%. Nothing was altered between the two models except the train/test split, in order to get a rough gauge of how the model would tend to perform. ResNet18 model 1 was trained using a 90/10 train/test split across both of its rounds. The first round was run over 10 epochs, and the second round over 20 epochs. The

train/test split was changed between the two rounds. The model achieved an overall accuracy of 99.3%.

Discussion and Conclusion

Based on these results, it's clear that CNNs are a fantastic choice for guitar effects classification. This is most highlighted by the fact that our simple VGG model achieved an accuracy above 90% despite only having 4 convolutional layers. Our highest accuracy being 99.3% accuracy is also a very good sign, though it's less interpretable given that we don't know what the model was pretrained on.

The results of this project can only be extended to the overdrive, distortion, and fuzz guitar effects. Datasets that include other effects (e.g., chorus, reverb, phaser) would have to be generated and validated to make definitive conclusions as to the spectrum of effects that are able to be modeled. This project also never utilized samples longer than 2 seconds, which could heavily impact the applications of this model. Future datasets would need to consider different sample sizes, different combinations of effects at once, and multiple guitars being played at once.

Of course, this project is not without its limitations. Given the time constraints, testing was very limited and not at all thorough. Future trials should be done using the K-Folds validation procedure so that the models are properly trained and validated across all of the available data. This is important because each sample represents a different unique combination of sounds, so if the models aren't trained on all the samples, key information could be missing that lets it generalize. This experiment could have also benefited from more systematized training organization and testing. In the future, I would modify the training functions such that they record key training data, such as loss and accuracy over each training, and I would make it so that you could train for longer sessions without needing to constantly monitor and record the results.

Different models with different numbers of layers could have also been tested. One interesting idea would be to add more and more convolutional layers to the simple VGG model and see how training loss and accuracy change over time across different numbers of epochs.

Additionally, other model architectures could be explored to further evaluate if there are pros and cons across models when classifying guitar effects.

The ethical and social implications of this project are somewhat limited. Given that this deals with audio, it would be possible to have a dataset that only contains recorded songs from public artists. This could be problematic depending on how the music is acquired and whether or not the artist is ok with their work being used in such projects. However, as showcased by the GFD dataset, through a lot of dedication and some scripting, it's possible to systematically create the data necessary to achieve a high level of accuracy. A model that could accurately classify any guitar pedal effect (or any sound for that matter) would be very useful for beginner artists wanting to recreate their favorite artist's style and tone.

Overall, this project highlights the key struggles of most machine learning projects, namely, data collection and organization. It took a long time to get the datasets organized and loaded onto Colab just to train the first model, and that was even when it was already thoroughly processed and organized by the original authors. It also highlights just how much data is required to train such models, and the limitations that impose on those who are trying to do exploratory analysis of such topics, both in terms of the amount of time it takes to process, train, and validate the models, and also in the computational resources necessary for it. Nevertheless, I believe this project answers at least the first research question it set out to explore, and showcases what is possible in machine learning now, even when all you have is a single laptop.

Sources

1. Bognár, P. (2022). Audio effect modeling with deep learning methods [Diploma Thesis, Technische Universität Wien]. reposiTUm. <https://doi.org/10.34726/hss.2022.94860>
2. Chulev, Joanikij. "Improving musical instrument classification with advanced machine learning techniques." arXiv preprint arXiv:2411.00275 (2024). <https://arxiv.org/abs/2411.00275>

3. de Castro Rabelo Profeta, R., Schuller, G. (2020). Feature-Based Classification of Electric Guitar Types. In: Cellier, P., Driessens, K. (eds) Machine Learning and Knowledge Discovery in Databases. ECML PKDD 2019. Communications in Computer and Information Science, vol 1168. Springer, Cham.
https://doi.org/10.1007/978-3-030-43887-6_41
4. F. Rong, "Audio Classification Method Based on Machine Learning," 2016 International Conference on Intelligent Transportation, Big Data & Smart City (ICITBS), Changsha, China, 2016, pp. 81-84, doi: 10.1109/ICITBS.2016.98. keywords: {Transportation;Big Data;Smart cities;Audio classification;Machine learning;SVM;Audio frame;Audio shot},
<https://ieeexplore.ieee.org/abstract/document/8047110>
5. "Resnet18 from Scratch Using Pytorch." *GeeksforGeeks*, GeeksforGeeks, 27 Nov. 2025, www.geeksforgeeks.org/deep-learning/resnet18-from-scratch-using-pytorch/.
6. Hinrichs, Reemt, et al. "Convolutional neural networks for the classification of guitar effects and extraction of the parameter settings of single and multi-guitar effects from instrument mixes." *EURASIP Journal on Audio, Speech, and Music Processing* 2022.1 (2022): 28.
7. Roberts , Leland. "Understanding the Mel Spectrogram ." *Medium*, Analytics Vidhya, 5 Mar. 2020, medium.com/analytics-vidhya/understanding-the-mel-spectrogram-fca2afa2ce53.
8. "VGG-Net Architecture Explained." *GeeksforGeeks*, GeeksforGeeks, 23 July 2025, www.geeksforgeeks.org/computer-vision/vgg-net-architecture-explained/.
9. Wasser, Leah A. "Hierarchical Data Formats - What Is HDF5?" *NEON SCIENCE*, 2 Apr. 2026, www.neonscience.org/resources/learning-hub/tutorials/about-hdf5.
10. Audio Signal Processing for Machine Learning:
<https://www.youtube.com/playlist?list=PL-wATfeyAMNglee7cH3q1bh4QJFAaeNv0>

Datasets

Marco Comunita'. GUITAR-FX-DIST: A Dataset of Processed Guitar Recordings for Music Research - (Mono Discrete). 1.0, Zenodo, 30 Nov. 2020, <https://doi.org/10.5281/zenodo.4298000>.

Kehling, C., et al. IDMT-SMT-Guitar Dataset. 1.0.0, Zenodo, 17 Jan. 2023, <https://doi.org/10.5281/zenodo.7544110>.